



✓ Introduction to Seaborn :-

Seaborn is a sophisticated Python data visualization library built on top of Matplotlib and closely integrated with Pandas data structures. It provides a high-level interface for creating attractive and informative statistical graphics. While Matplotlib provides the core rendering engine, Seaborn abstracts away the boilerplate code required for complex tasks like plotting multi-dimensional data, performing automatic statistical aggregation (such as mean and confidence intervals), and managing sophisticated color palettes.

By leveraging the power of Matplotlib while offering a more intuitive API, Seaborn has become the industry standard for Exploratory Data Analysis (EDA). It allows data scientists to focus on understanding the underlying patterns in their data rather than the mechanics of the plotting library.

History :-

Seaborn was originally developed by Michael Waskom, a researcher at New York University and Stanford, around 2012. The project was born out of a necessity to address several limitations in the early versions of Matplotlib: primarily the lack of high-level statistical functions, the verbose code required for data grouped by multiple variables, and the outdated default aesthetic styles.

Over the last decade, Seaborn has evolved from a small utility for Waskom's own research into a cornerstone of the Python data science ecosystem. It is now part of the NumFOCUS sponsored projects and continues to be actively maintained, with

recent versions introducing the 'objects' interface—a new declarative system for composing plots that aligns more closely with the 'Grammar of Graphics' philosophy found in libraries like ggplot2.

Why Seaborn is Developed :-

Seaborn was developed to bridge the gap between simple plotting and complex statistical analysis. It addresses the need for a library that can understand the structure of data and provide a concise syntax for common visualization tasks.

Key motivations and parameters behind its development include:

1. **Integration with Pandas:** Seaborn is designed to work seamlessly with `DataFrame` objects, using column names as axis labels and legend titles automatically.
2. **Statistical Estimation:** Unlike Matplotlib, Seaborn automatically performs statistical aggregation and plots error bars (representing confidence intervals) using parameters like `ci` or `errorbar`.
3. **Semantic Mapping:** It uses parameters like `hue`, `size`, and `style` to map data variables to visual properties, allowing for the visualization of up to four or five dimensions in a single 2D plot.
4. **Aesthetic Defaults:** It was built to replace the basic Matplotlib defaults with high-quality, professional themes through the `set_theme()` function.

Features :-

Key features include:

- Functions for visualizing univariate and bivariate distributions.
- Automatic estimation and plotting of linear regression models.
- High-level abstractions for multi-plot grids (FacetGrid).
- Support for categorical data plotting.

Applications :-

Seaborn is extensively used in:

- Exploratory Data Analysis (EDA).
- Statistical modeling and hypothesis testing.
- Academic research and publication-quality reporting.
- Feature engineering and outlier detection.

✓ Installation and Importing Seaborn :-

Installation

- **pip install:** The most common way to install the latest stable version from PyPI.
- **Dependencies:** Seaborn will automatically install necessary dependencies like NumPy, Pandas, and Matplotlib if they are missing.

```
# Step 1: Install the library
!pip install seaborn
```

✓ Importing

- **as sns:** It is a standard convention in the Python community to alias Seaborn as `sns`.
- **set_theme:** A global command to apply Seaborn's aesthetic defaults to all subsequent plots.

```
import matplotlib.pyplot as plt
import seaborn as sns
```

✓ Built-in Dataset (load_dataset) :-

Seaborn comes with several built-in datasets for practice. The `load_dataset` function allows you to fetch these as Pandas DataFrames instantly.

```
tips = sns.load_dataset("tips")
display(tips.head())
```

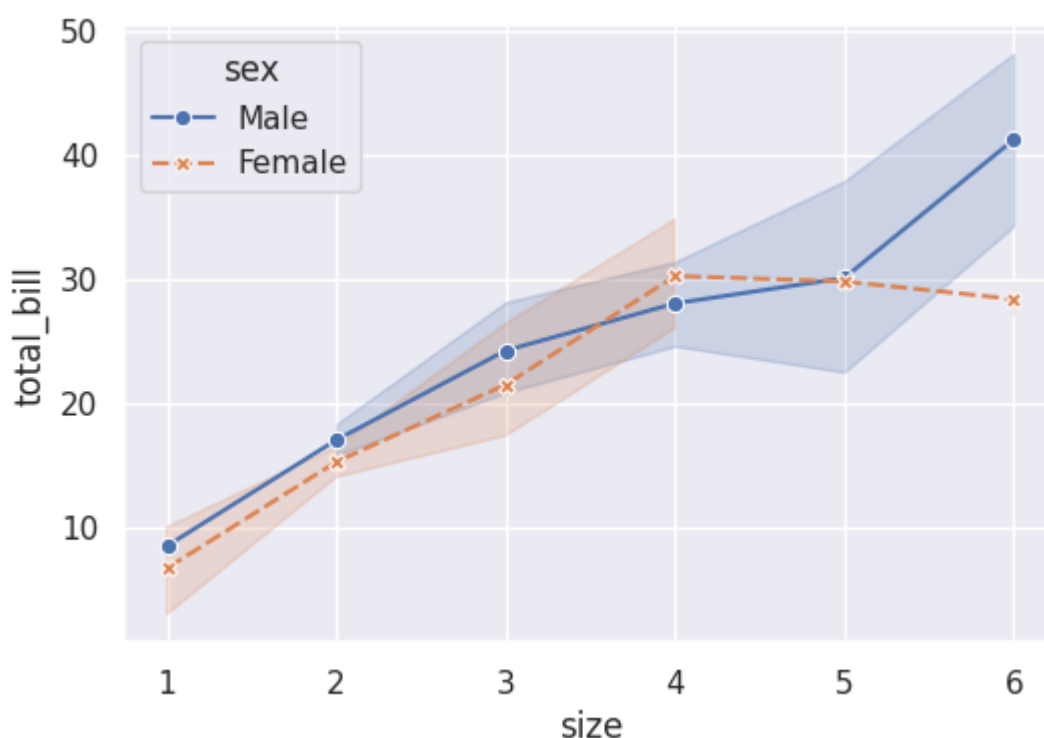
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

✓ Line Plot :-

Line plots are essential for visualizing trends over continuous intervals, such as time. Below are the parameters used to control the visualization:

- **data**: The source dataset (Pandas DataFrame).
- **x / y**: The variables to be plotted on the horizontal and vertical axes.
- **hue**: Groups the data by a categorical variable and assigns different colors.
- **markers**: If True, adds distinct symbols to each data point for better visibility.
- **style**: Assigns different line styles (solid, dashed, etc.) based on a categorical variable.

```
plt.figure(figsize=(6, 4))
sns.lineplot(data=tips, x="size", y="total_bill", hue="sex", markers=True, style="sex")
plt.show()
```

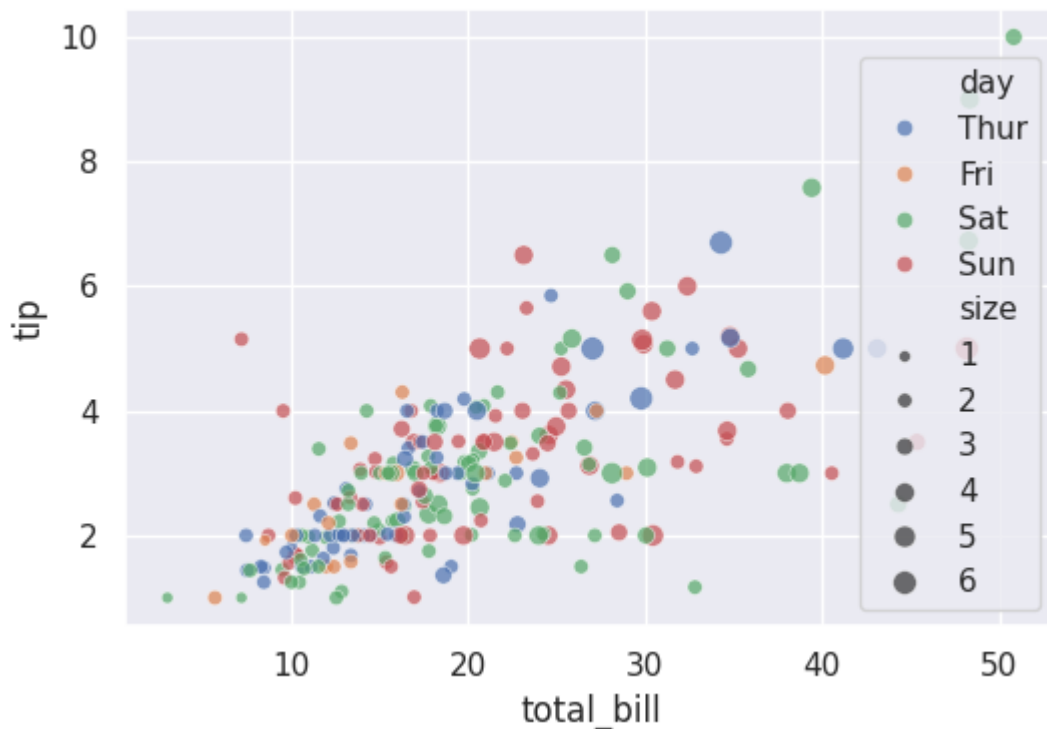


✓ Scatter Plot :-

Scatter plots are used to observe relationships between two numeric variables. Here is a breakdown of the parameters used:

- **data**: The source dataset.
- **x / y**: The numeric variables for coordinates.
- **size**: Scales the diameter of the markers based on a numeric variable (e.g., table size).
- **alpha**: Sets the transparency level (0 to 1) to help visualize overlapping points.
- **hue**: Colors the points based on a specific category (e.g., day of the week).

```
plt.figure(figsize=(6, 4))
sns.scatterplot(data=tips, x="total_bill", y="tip", size="size", alpha=0.7,
plt.show())
```



✓ Bar Plot :-

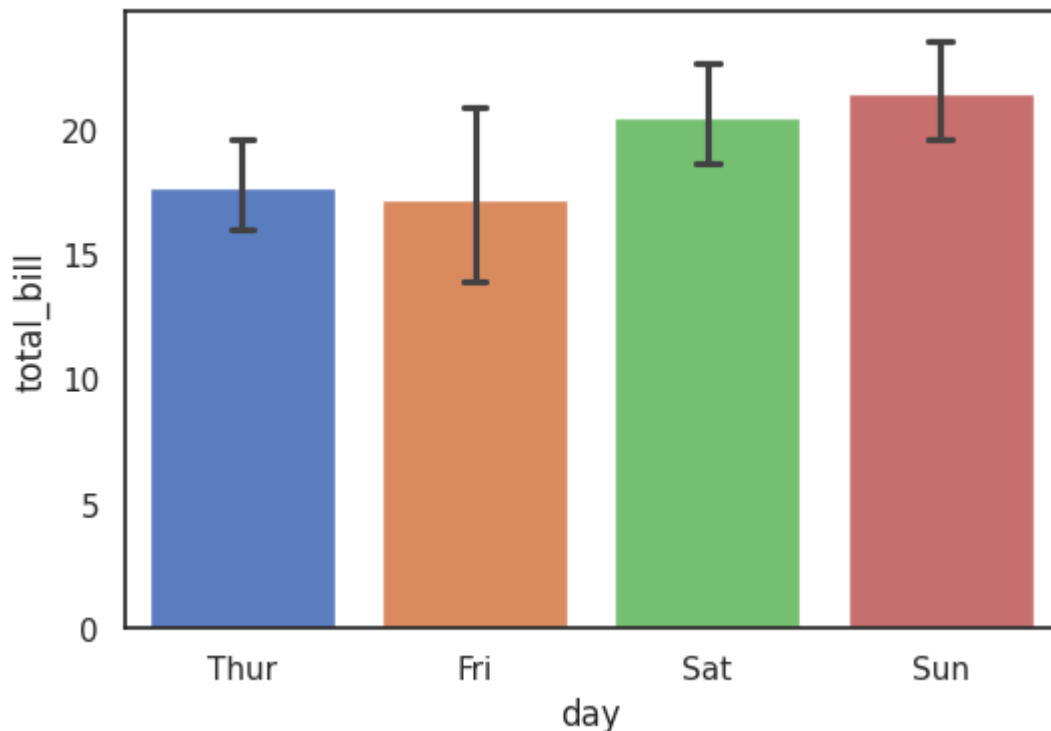
Bar plots represent an estimate of central tendency for a numeric variable. We utilize the following parameters:

- **data:** The source DataFrame.
- **x / y:** The categorical and numeric variables respectively.
- **palette:** Defines the specific color mapping for the categorical levels.
- **capsize:** Determines the width of the caps on the error bars to make them more distinct.
- **hue:** (Recommended fix) Assigning the x-variable to hue avoids modern deprecation warnings.

```
plt.figure(figsize=(6, 4))
sns.barplot(data=tips, x="day", y="total_bill", palette="muted", capsize=0.1
plt.show())
```

```
/tmp/ipykernel_736/1026664798.py:2: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed i
sns.barplot(data=tips, x="day", y="total_bill", palette="muted", capsize=0.
```

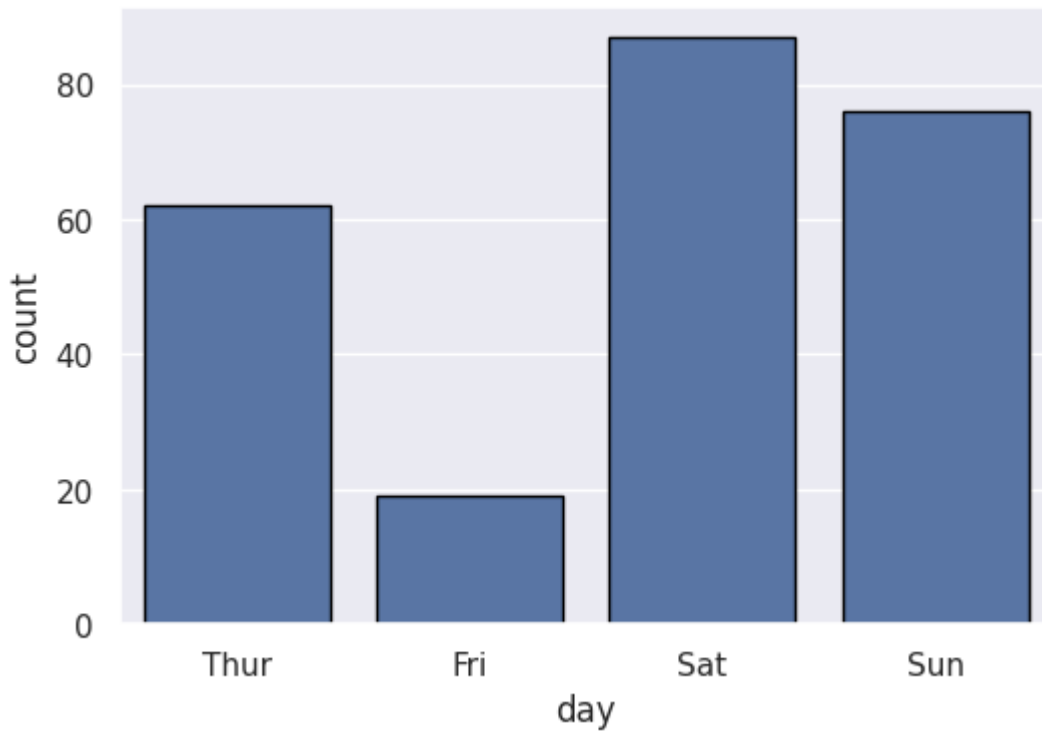


✓ Count Plot :-

A count plot shows the frequency of observations in each categorical bin. Parameters used include:

- **data**: The source dataset.
- **x**: The categorical variable to count.
- **order**: Manually specifies the sequence of categories displayed on the axis.
- **edgecolor**: Sets the color of the border around each bar for sharper visual definition.

```
plt.figure(figsize=(6, 4))
sns.countplot(data=tips, x="day", order=["Thur", "Fri", "Sat", "Sun"], edgec
plt.show()
```

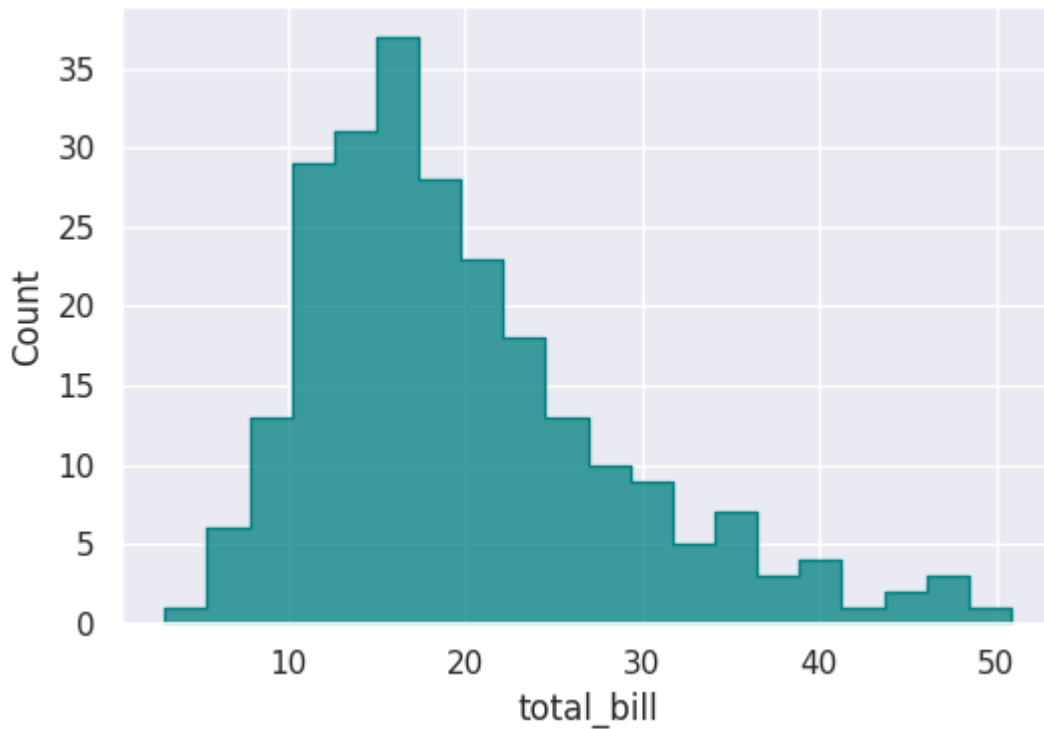


✓ Histogram :-

Histograms examine the distribution of a single quantitative variable. Key parameters include:

- **data:** The source dataset.
- **x:** The numeric variable for the distribution.
- **bins:** Controls the number of equal-width bins (intervals) used to group the data.
- **element:** Changes the visual style of the bars (e.g., 'step' creates an outline rather than filled bars).
- **color:** Explicitly sets the fill color for the distribution.

```
plt.figure(figsize=(6, 4))
sns.histplot(data=tips, x="total_bill", bins=20, element="step", color="teal")
plt.show()
```

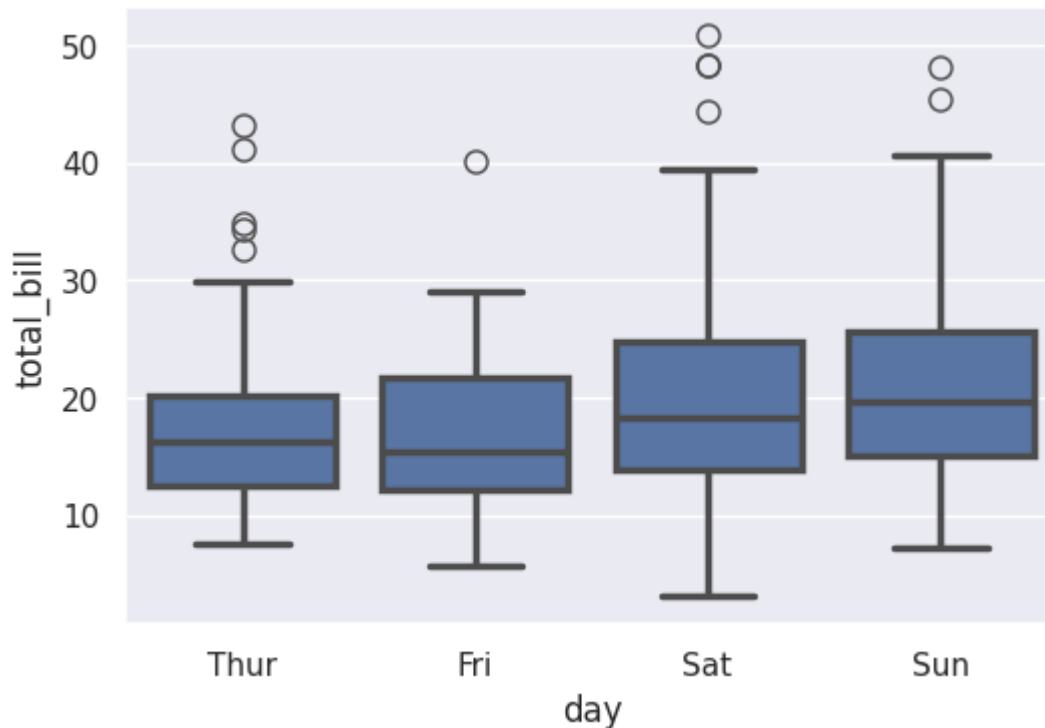


✓ Box Plot :-

Box plots provide a summary of data through quartiles. The following parameters enhance the plot:

- **data:** The source dataset.
- **x / y:** Variables for grouping and the numeric range.
- **linewidth:** Adjusts the thickness of the box, median, and whisker lines.
- **fliersize:** Adjusts the size of the markers used to represent statistical outliers.

```
plt.figure(figsize=(6, 4))
sns.boxplot(data=tips, x="day", y="total_bill", linewidth=2.5, fliersize=8)
plt.show()
```

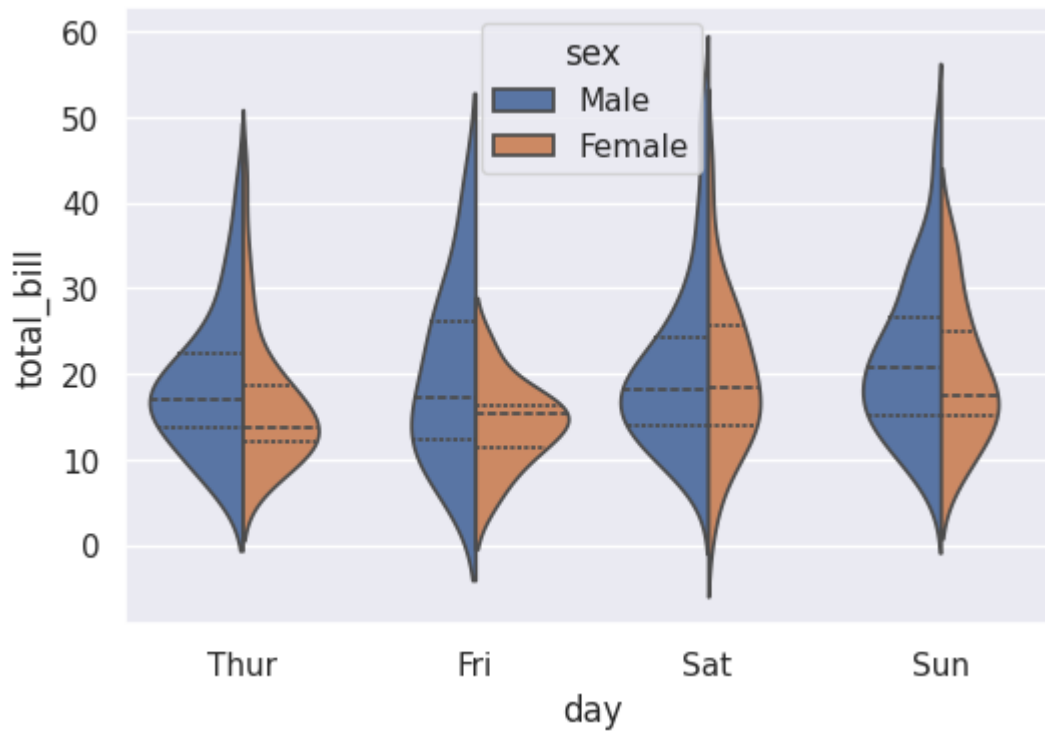



✓ Violin Plot :-

Violin plots combine box plots with kernel density estimation. The parameters used are:

- **data:** The source dataset.
- **x / y:** The categorical and numeric axes.
- **hue:** Splits the data further into sub-groups (e.g., Male vs Female).
- **split:** When True, draws half a violin for each hue level, making comparison easier.
- **inner:** Determines how the internal data is represented (e.g., 'quart' shows the quartiles).

```
plt.figure(figsize=(6, 4))
sns.violinplot(data=tips, x="day", y="total_bill", hue="sex", split=True, inner="quart")
plt.show()
```

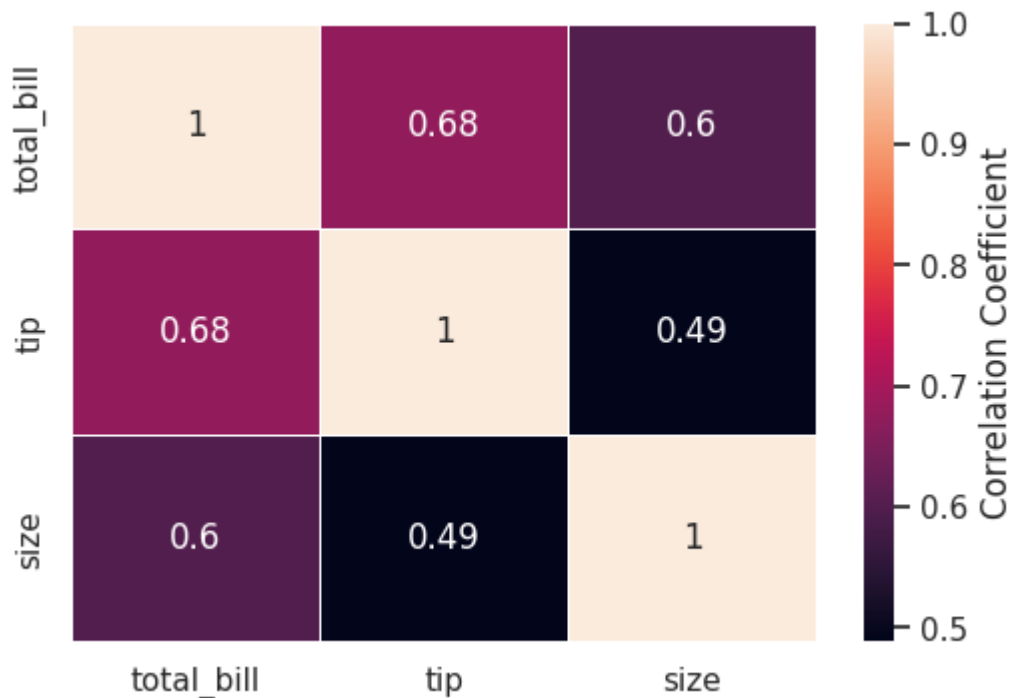


✓ Heatmap :-

Heatmaps visualize matrix data. Here are the core parameters applied:

- **data:** A matrix or correlation table (numeric values).
- **annot:** When True, writes the data value in each cell.
- **linewidths:** Adds thin spaces between each cell for visual separation.
- **cbar_kws:** A dictionary of keyword arguments for the color bar, such as adding a label.

```
corr = tips.select_dtypes('number').corr()
plt.figure(figsize=(6, 4))
sns.heatmap(corr, annot=True, linewidths=.5, cbar_kws={'label': 'Correlation'})
plt.show()
```



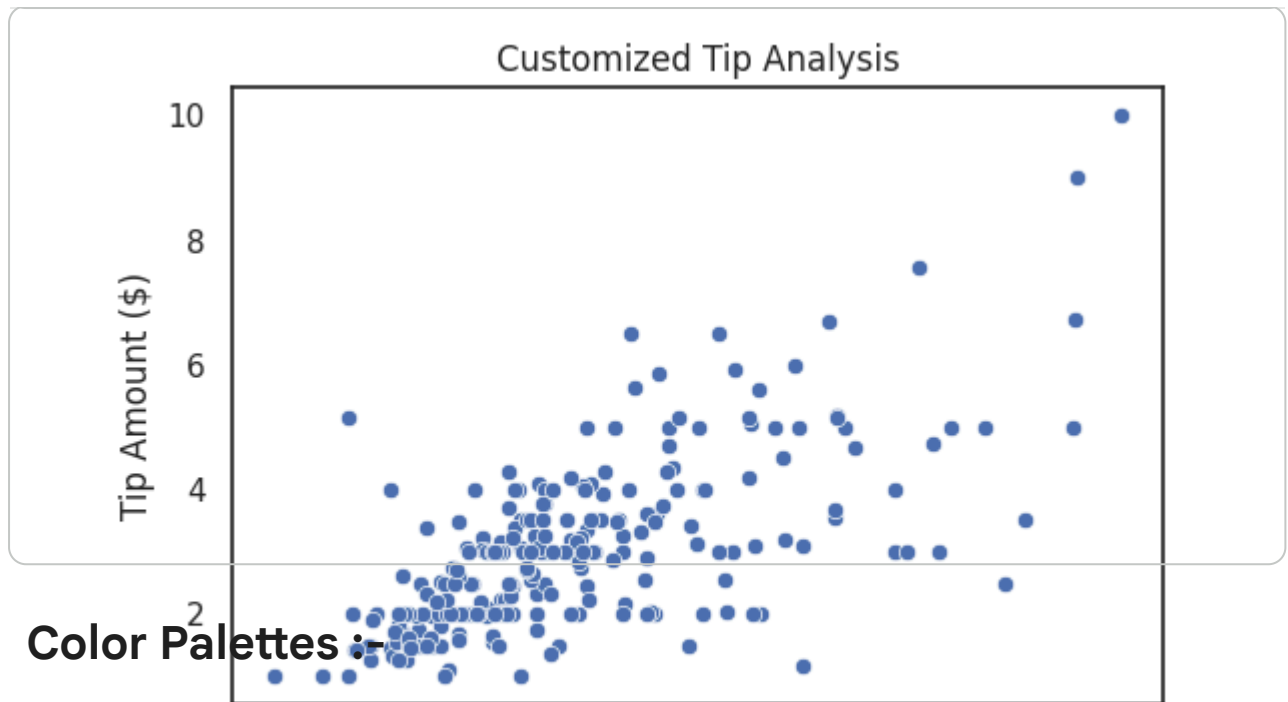
✓ Graph Customization :-

Seaborn allows you to control the look of your plots using high-level style commands and standard Matplotlib functions.

Here are the parameters and functions used to customize the visualization:

- **style (sns.set_style):** Determines the background theme of the plot (e.g., 'white', 'darkgrid', 'ticks').
- **data:** The source dataset used for the scatter plot.
- **x / y:** The variables mapped to the horizontal and vertical axes.
- **plt.title:** Sets the main heading of the chart to provide context.
- **plt.xlabel / plt.ylabel:** Customizes the labels for the axes to make the units or variables clear.

```
sns.set_style("white")
plt.figure(figsize=(6, 4))
sns.scatterplot(data=tips, x="total_bill", y="tip")
plt.title("Customized Tip Analysis")
plt.xlabel("Total Bill ($)")
plt.ylabel("Tip Amount ($)")
plt.show()
```



✓ Color Palettes :-

Seaborn offers a range of built-in color palettes to make your plots visually appealing and meaningful. Choosing the right palette is crucial for distinguishing between categories or showing the intensity of values.

Here are the functions and parameters used in the code below:

- **sns.color_palette**: A function that returns a list of RGB tuples defining a color scheme (e.g., 'viridis', 'magma', 'Blues').
- **sns.palplot**: A convenience function that plots the values in a color palette as a horizontal array of colors for quick previewing.

```
sns.palplot(sns.color_palette("viridis"))
sns.palplot(sns.color_palette("magma"))
sns.palplot(sns.color_palette("Blues"))
```

